

평행 주행 오경보 개선 — DCPA 게이트 적용 결과

AI-V2X 스마트 지팡이 · SUMO/AI 파트 · 2026. 8. 5 · 적용 위치: Jetson 자동 추론 파이프라인 (process_scenarios.py)

1. 발견한 문제 — "옆으로 지나가는 차도 위험이라고 한다"

AI 추론 결과를 검증하던 중, 보행자와 평행하게 달리거나 스쳐 지나가는 차량이 위험(레벨 2~3)으로 판정되는 현상을 발견. 실제 데이터로 확인한 결과:

- 위험(L2+) 판정 차량 중 58.7%는 보행자 10m 이내로 접근한 적조차 없음 (40개 시나리오, 4,439대 분석)
- 실제 최소 접근 거리가 37m인 차량이 '매우 위험(L3)'으로 판정된 사례도 존재
- 규칙 채점표로 판정해도 동일한 현상(64.7%) → 모델 문제가 아니라 특징(입력값)의 문제

2. 원인 — 스칼라 값은 방향을 모른다

현재 모델 입력(거리, 접근속도, TTC)은 모두 크기만 있는 스칼라 값. 옆 차선으로 지나갈 차도 가장 가까운 지점에 도달하기 전까지는 거리가 계속 줄어들기 때문에, 접근속도(+)-TTC(작음) 등 숫자만 보면 정면으로 돌진하는 차와 구분이 안 된다. "거리가 줄고 있다"는 사실만으로는 부딪힐 차인지 스쳐 갈 차인지 알 수 없다.

3. 해결 — 벡터로 "최근접 예상 거리(DCPA)"를 예측

- ① 차량-보행자의 상대 위치 벡터와 상대 속도 벡터를 매초 계산
- ② "이 방향·속도를 유지하면 보행자와 최소 몇 m까지 가까워지는가" = DCPA 예측
- ③ 팀 게이트 공식(step7_risk.py의 dcpa_gate 그대로 재사용)으로 위험 레벨 억제:
 - DCPA 2.5m 이내 (충돌 가능 코스) → 억제 없음, 그대로 경보
 - DCPA 7.5m 이상 (명백히 빗나감) → 점수 80% 억제
 - 사이 구간 → 선형 감소 · 게이트는 레벨을 올리지는 않음(억제 전용)
- ④ 결과 CSV에 dcpa_m · gate_factor · 게이트 전 원본 판정을 함께 저장 → 언제든지 재검증 가능

4. 적용 전후 비교 (82개 시나리오 · 차량 9,076대)

지표	게이트 전	게이트 후	변화
L2+ 판정 중 10m 이내로 안 온 차 (오경보율)	58.7%	18.1%	▼ 69% 감소
L2+ 판정 중 20m 이내로도 안 온 차	41.5%	8.7%	▼ 79% 감소
위험(L2+) 판정 차량 수	2,762대	847대	불필요한 경보 69% 감소

안전 검증 — 진짜 위험은 하나도 놓치지 않음
실제로 보행자 5m 이내까지 접근한 차량 398대 전원, 게이트 적용 후에도 위험(L2+) 판정 유지 — 놓친 차량 0대. 스쳐 가는 차만 걸러내고 충돌 코스는 전부 보존.

5. 남은 근본 과제

- 이번 게이트는 후처리 방식 — 예정된 모델 재학습 때 상대 위치·속도 벡터를 모델 입력에 직접 추가하면 게이트 없이도 모델이 스스로 방향을 구분
- 라벨을 만드는 채점표에도 같은 한계가 있으므로, 재학습 시 라벨 생성 기준에도 DCPA 반영 ("위험 정의 일원화" 과제와 연결)
- 현재 Jetson에 쌓인 시나리오 1,185개 전체를 새 기준으로 재추론 중 — 완료되면 위험지도도 자동으로 갱신됨

요약: 오경보 59% → 18%로 줄이면서 실제 위험 검출은 100% 유지. 발견 → 데이터 검증 → 원인 규명 → 팀 로직 재사용 → 정량 검증의 절차로 진행함.